

A Practical Course on Mathematical Modelling

Lecture 1 - An overview of Mathematical Optimization

Prof. Viet Hung Nguyen

Clermont Auvergne University - Clermont-Ferrand - France

- Ph.D in Computer Science at Aix-Marseille University, France
- Habilitation à Diriger des Recherche at Sorbonne University, France
- Assistant & Associate Professor at Sorbonne University for 16 years.
- Full Professor at Clermont Auvergne University, Clermont Ferrand in the center of France since 2018.
- Head of Combinatorial Optimization at LIMOS lab (<https://www.limos.fr>)
- Head of International Master of Computer Science Program at ISIMA, Clermont Auvergne University (www.isima.fr)

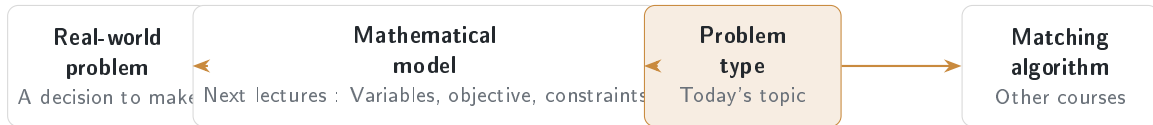
Outline of the course

- Lecture 1: Optimization overview
- Lecture 2: Julia/JuMP programming
- Lecture 3: Linear Programming models
- Lecture 4: Integer Programming models
- Lecture 5: Combinatorial Optimization: models and algorithms
- Lecture 6: Stochastic and robust programming
- Lecture 7: Project

Classification is the first modelling decision

WHY CLASSIFY

Algorithms for solving optimization problems are tailored to a particular type of problem. Before choosing a solver, you classify the model — that choice determines which methods even apply.



FOUR AXES OF OPTIMIZATION PROBLEMS

Convex vs **Nonconvex**

Does a local minimum guarantee a global one?

Continuous vs **Discrete**

Can variables take any real value, or only some?

Unconstrained vs **Constrained**

Are there explicit limits on the variables?

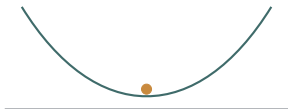
Deterministic vs **Stochastic**

Is the problem data known accurately, or uncertain?

Convex vs. Nonconvex

AXIS 1

Convex



Any local minimum is a global minimum. Structure you can exploit: gradient-based methods converge reliably.

Nonconvex



Multiple local minima can trap an algorithm. Finding the global optimum (gold) is generally much harder.

Refresher: Derivative, Gradient, Hessian

THE TOOLS OF DIFFERENTIAL CALCULUS

Case $n = 1$

First derivative $f'(x) \in \mathbb{R}$

Slope of the tangent line — direction and rate of change of f at x .

Second derivative $f''(x) \in \mathbb{R}$

Curvature of f at x — local convexity ($f'' > 0$) or local concavity ($f'' < 0$).

General case $n \geq 1$

Gradient $\nabla f(x) \in \mathbb{R}^n$

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x), \dots, \frac{\partial f}{\partial x_n}(x) \right)^\top$$

Vector of partial derivatives — generalizes $f'(x)$; points in the direction of steepest ascent.

Hessian $\nabla^2 f(x) \in \mathbb{R}^{n \times n}$

$$\nabla^2 f(x) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}(x) \right]_{i,j=1}^n$$

Matrix of second derivatives — generalizes $f''(x)$; symmetric whenever f is of class C^2 .

Gradient and Hessian — Worked Examples

PUTTING THE DEFINITIONS TO WORK

$$f(x, y) = x^2 + y^2$$

$$\text{Partial derivatives: } \frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 2y$$

$$\text{Gradient: } \nabla f(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$$

$$\text{Second partials: } \frac{\partial^2 f}{\partial x^2} = 2, \quad \frac{\partial^2 f}{\partial x \partial y} = 0,$$

$$\frac{\partial^2 f}{\partial y^2} = 2$$

$$\text{Hessian: } \nabla^2 f(x, y) = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$$

$$f(x, y) = x^2$$

$$\text{Partial derivatives: } \frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 0$$

$$\text{Gradient: } \nabla f(x, y) = \begin{pmatrix} 2x \\ 0 \end{pmatrix}$$

$$\text{Second partials: } \frac{\partial^2 f}{\partial x^2} = 2, \quad \frac{\partial^2 f}{\partial x \partial y} = 0,$$

$$\frac{\partial^2 f}{\partial y^2} = 0$$

$$\text{Hessian: } \nabla^2 f(x, y) = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$$

These are exactly the two functions used to illustrate strict vs. non-strict convexity a few slides ahead.

Convex Functions

AXIS 1 · DEFINING CONVEXITY

A function f is **convex** on a convex domain if, using the tools above:

First-order condition

For all x, y in the domain,

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x)$$

The tangent (hyper)plane at any point lies everywhere below the graph of f .

Second-order condition (if f is C^2)

$$\nabla^2 f(x) \succeq 0 \text{ for all } x \quad (f''(x) \geq 0 \text{ everywhere, if } n = 1)$$

The Hessian is positive semidefinite everywhere — f never curves downward. Strict positive definiteness ($\succ 0$) gives strict convexity.

Convex Functions — Examples in 1D

AXIS 1 · EXAMPLES IN DIMENSION 1

Unimodal, not convex



$f(x) = \sqrt{|x|}$: a single global minimum at $x = 0$ — decreasing then increasing, i.e. **unimodal** — yet f is concave on each branch, so f is **not convex**.

Convex, not unimodal

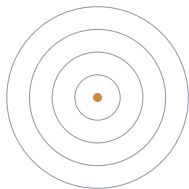


$f(x) = \max(0, |x| - 1)$: **convex** (a max of affine functions), but flat — and thus minimized — on the whole interval $[-1, 1]$: no single minimizer, so **not unimodal** in the strict sense.

Convex Functions — Examples in 2D

AXIS 1 · EXAMPLES IN DIMENSION 2

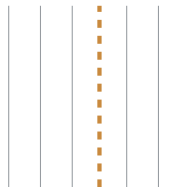
Strictly convex



$$f(x, y) = x^2 + y^2, \quad \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \succ 0$$

everywhere: level sets are circles shrinking to a point
— a **unique** global minimizer.

Convex, not strict



$$f(x, y) = x^2, \quad \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix} \succeq 0 \text{ but not } \succ 0:$$

flat (zero curvature) along y — the whole line $x = 0$
(gold) minimizes f .

AXIS 1 · KEEPING THE FEASIBLE SET CONVEX

The feasible set $\{x : g_i(x) \leq 0, i \in \mathcal{I}, h_j(x) = 0, j \in \mathcal{E}\}$ is convex whenever:

- **Inequality constraints** $g_i(x) \leq 0$ are **convex**: each g_i is a convex function (as defined above), so its sublevel set $\{x : g_i(x) \leq 0\}$ is convex.
- **Equality constraints** $h_j(x) = 0$ are **affine**: $h_j(x) = a_j^\top x - b_j$. A curved equality constraint would carve out a non-convex set, even if h_j itself is convex.

Convex problem: minimize a convex f over a convex feasible set — convex objective, convex inequality constraints, affine equality constraints. Every local minimum is then global.

Convex Constraint — Example

AXIS 1 · A CONVEX FEASIBLE SET

Take the constraint

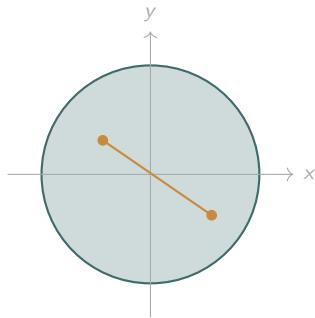
$$g(x, y) = x^2 + y^2 - 1 \leq 0.$$

g is convex (same $f(x, y) = x^2 + y^2$ from a few slides ago, shifted by a constant — shifting never changes convexity, since $\nabla^2 g = \nabla^2 f \succ 0$ everywhere).

Its feasible set is the sublevel set

$$\{(x, y) : g(x, y) \leq 0\} = \{(x, y) : x^2 + y^2 \leq 1\},$$

the closed unit disk — a **convex** set: the segment between any two feasible points stays feasible.



The segment joining two points of the disk never leaves it.

Axis 2: Unconstrained vs. Constrained Optimization

A NEW AXIS OF CLASSIFICATION

Unconstrained vs Constrained

Are there explicit limits on the variables?

In this section:

- Unconstrained optimization, and its optimality conditions
- Worked examples: local vs. global minima, and a stationary point that isn't optimal
- Constrained optimization and the KKT conditions
- KKT points that are optimal — and ones that aren't

AXIS 2 · NO CONSTRAINTS

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \ f(x)$$

Where it comes from

Arises directly in many applications, and indirectly when a constrained problem's constraints are folded into the objective as a penalty term.

What's required

f is a smooth function on $\mathbb{R}^n$; x is a real vector with $n \geq 1$ components. No restriction beyond that.

Optimality Conditions — Unconstrained Case (1/2)

WHEN IS x^* A LOCAL MINIMUM? — NECESSARY CONDITIONS

First-order necessary condition

If x^* is a local minimum of f and f is differentiable at x^* , then

$$\nabla f(x^*) = 0 \quad (f'(x^*) = 0 \text{ if } n = 1)$$

x^* is then called a stationary point (or critical point) — necessary but not sufficient.

Second-order necessary condition

If, in addition, f is twice differentiable at x^* ,

$$\nabla^2 f(x^*) \succeq 0 \quad (f''(x^*) \geq 0 \text{ if } n = 1)$$

The Hessian must be positive semidefinite (no direction of negative curvature).

Optimality Conditions — Unconstrained Case (2/2)

WHEN ARE WE SURE x^* IS A MINIMUM? — SUFFICIENT CONDITION

Second-order sufficient condition

If

$$\nabla f(x^*) = 0 \quad \text{and} \quad \nabla^2 f(x^*) \succ 0$$

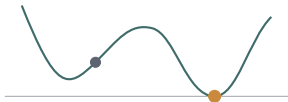
(that is, $f'(x^*) = 0$ and $f''(x^*) > 0$ if $n = 1$), then x^* is a strict local minimum of f .

- Stationary + Hessian **positive** definite $\Rightarrow$ strict local minimum.
- Stationary + Hessian **negative** definite $\Rightarrow$ strict local maximum.
- Stationary + Hessian **indefinite** (eigenvalues of opposite signs) $\Rightarrow$ saddle point.

Unconstrained Optimization — Examples

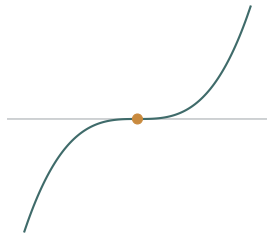
AXIS 2 · LOCAL VS. GLOBAL, STATIONARY VS. OPTIMAL

Local min $\neq$ global min



$f(x) = (x^2 - 1)^2 + 0.3x$: both marked points satisfy $f' = 0$, $f'' \geq 0$ — two local minima — but only the deeper one (gold) is the **global** minimum.

Stationary but not optimal



$f(x) = x^3$: $f'(0) = 0$ is stationary, but $f''(0) = 0$ too — the 2nd-order test is inconclusive, and $x = 0$ is in fact **neither** a local min nor a local max (an inflection point).

AXIS 2 · WITH CONSTRAINTS

$$\begin{array}{ll}\text{minimize} & f(x) \\ \text{subject to} & h_i(x) = 0, \quad \forall i \in \mathcal{E} \\ & g_i(x) \leq 0, \quad \forall i \in \mathcal{I}\end{array}$$

f , the h_i , and the g_i are smooth, real-valued functions;
 $\mathcal{E}$ and $\mathcal{I}$ index the equality and inequality constraints.

The feasible set

The set of points x that satisfy every constraint. Constrained problems are further classified by the nature of the constraints — linear, nonlinear, convex — and the smoothness of the functions.

Constraints range from simple bounds to full systems of equalities and inequalities modelling complex relationships among variables.

Optimality Conditions — Constrained Case (KKT)

KARUSH-KUHN-TUCKER

We introduce the **Lagrangian**, with one multiplier λ_i per equality constraint and $\mu_i \geq 0$ per inequality constraint:

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_{i \in \mathcal{E}} \lambda_i h_i(x) + \sum_{i \in \mathcal{I}} \mu_i g_i(x)$$

Under a constraint qualification, if x^* is a local minimum, there exist λ^*, μ^* such that:

- **Stationarity:** $\nabla_x \mathcal{L}(x^*, \lambda^*, \mu^*) = 0$
- **Primal feasibility:** $h_i(x^*) = 0 \ \forall i \in \mathcal{E}, \quad g_i(x^*) \leq 0 \ \forall i \in \mathcal{I}$
- **Dual feasibility:** $\mu_i^* \geq 0 \ \forall i \in \mathcal{I}$
- **Complementary slackness:** $\mu_i^* g_i(x^*) = 0 \ \forall i \in \mathcal{I}$

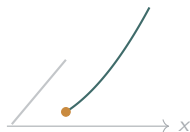
These generalize $\nabla f(x^*) = 0$ to the constrained case; the multipliers μ_i^* measure the sensitivity of f^* to relaxing constraint i .

KKT Points — Optimal or Not?

KKT IS NECESSARY, NOT ALWAYS SUFFICIENT

A KKT point that is optimal

minimize x^2 s.t. $g(x) = 1 - x \leq 0$



$x^* = 1$, $\mu^* = 2$: stationarity $2x^* - \mu^* = 0$, compl. slack. $\mu^*(1 - x^*) = 0$, $\mu^* \geq 0$ all hold.

f convex and g affine $\Rightarrow$ the problem is **convex**: the KKT point is guaranteed to be the **global minimizer**.

A KKT point that is not optimal

minimize $-x^2$ s.t. $-1 \leq x \leq 1$



$x = 0$, $\mu_1 = \mu_2 = 0$: constraints inactive, stationarity $f'(0) + 0 + 0 = 0$ holds trivially — a valid KKT point.

f is **concave** here: the problem is **non-convex**. $x = 0$ (muted) is in fact the local *maximum*; the true minimizers (gold) sit at the boundary $x = \pm 1$.

Continuous vs. Discrete

AXIS 3

Continuous

Variables take any value in a range, usually the reals. Smoothness lets objective and constraint values at a point predict a whole neighborhood — generally easier to solve.

Discrete

Some or all variables are binary, integer, or drawn from a finite set of objects. Harder in general, but advances in algorithms and computing have pushed the achievable scale dramatically.

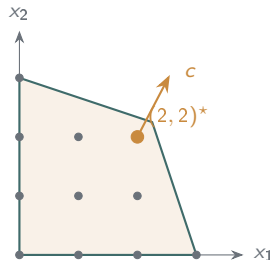
The link: continuous optimization is a building block for discrete optimization — many discrete algorithms solve a sequence of continuous subproblems.

Integer Linear Program — a 2D Example

DISCRETE OPTIMIZATION, WORKED EXAMPLE

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{s.t.} & x_1 + 3x_2 \leq 9 \\ & 3x_1 + x_2 \leq 9 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in \mathbb{Z}\end{array}$$

Structurally identical to a linear program
(same objective, same linear constraints), with
one added requirement: x_1, x_2 must be
integers.



Dots: the 11 feasible integer points. Best one found by inspection: $(2, 2)$, $z^* = 4$.

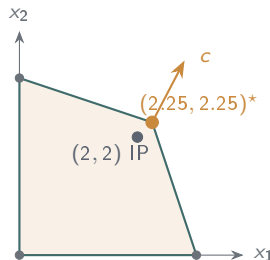
Its LP Relaxation

DROP INTEGRALITY, WORKED EXAMPLE CONTINUED

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{s.t.} & x_1 + 3x_2 \leq 9 \\ & 3x_1 + x_2 \leq 9 \\ & x_1, x_2 \geq 0\end{array}$$

Same feasible polygon, but the integrality requirement is dropped: x_1, x_2 may take any real value in it.

The LP relaxation's optimal value is always an **upper bound** on the integer program's (for a maximization): $4.5 \geq 4$. Rounding the relaxed solution does not always give the integer optimum — here it happens to.



$z_{LP}^* = 4.5$ at $(2.25, 2.25)$ — fractional, so not itself a valid integer solution.

Linear Programming: Canonical Form

FROM TAXONOMY TO PRACTICE

A linear program is a special case of the four-axis taxonomy introduced earlier: continuous, constrained, and both the objective and the constraints are linear.

$$\begin{array}{ll}\text{maximize} & c^T x \\ \text{subject to} & Ax \leq b \\ & x \geq 0\end{array}$$

$$c \in \mathbb{R}^n, \quad A \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m, \quad x \in \mathbb{R}^n.$$

- The feasible set $\{x : Ax \leq b, x \geq 0\}$ is a **polyhedron** — an intersection of finitely many half-spaces.
- If an optimum exists, it is attained at a **vertex** (an extreme point) of that polyhedron.

The Dual Program

ONE PROBLEM, TWO VIEWS

Primal (P)

$$\begin{array}{ll}\max & c^\top x \\ \text{s.t.} & Ax \leq b \\ & x \geq 0\end{array}$$

Dual (D)

$$\begin{array}{ll}\min & b^\top y \\ \text{s.t.} & A^\top y \geq c \\ & y \geq 0\end{array}$$

One dual variable $y_i \geq 0$ per primal constraint; one dual constraint per primal variable.

- **Weak duality:** for any feasible x and y , $c^\top x \leq b^\top y$.
- **Strong duality:** if (P) has an optimal solution, so does (D), and their optimal values coincide.

Complementary Slackness (LP)

LINKING PRIMAL, DUAL, AND KKT

For x feasible in (P) and y feasible in (D), both are optimal *iff*, for every i and every j ,

$$y_i(b_i - (Ax)_i) = 0 \quad \text{and} \quad x_j((A^\top y)_j - c_j) = 0$$

- $y_i > 0 \Rightarrow$ constraint i of (P) is **tight**:
 $(Ax)_i = b_i$.
- $x_j > 0 \Rightarrow$ constraint j of (D) is **tight**:
 $(A^\top y)_j = c_j$.
- Slack on one side forces the matching variable on the other side to be zero.

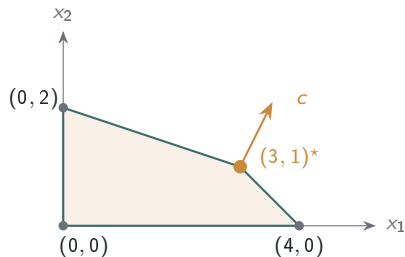
Link with KKT: write (P) as *minimize* $-c^\top x$ s.t. $Ax - b \leq 0$ (multiplier $\mu = y \geq 0$) and $-x \leq 0$ (multiplier $s \geq 0$). Stationarity gives $s = A^\top y - c$; KKT complementary slackness $\mu_i g_i(x^*) = 0$ and $s_j(-x_j^*) = 0$ is exactly the two boxed conditions.

A 2D Example — the Primal

WORKED EXAMPLE

$$\begin{array}{ll}\max & x_1 + 2x_2 \\ \text{s.t.} & x_1 + x_2 \leq 4 \\ & x_1 + 3x_2 \leq 6 \\ & x_1, x_2 \geq 0\end{array}$$

Two variables, two constraints — small enough to see the whole polyhedron.

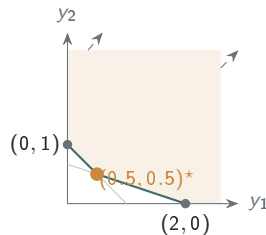


$$z(0, 0) = 0 \quad z(4, 0) = 4 \quad z(0, 2) = 4 \quad z(3, 1) = 5 \star$$

WORKED EXAMPLE, CONTINUED

$$\begin{array}{ll}\min & 4y_1 + 6y_2 \\ \text{s.t.} & y_1 + y_2 \geq 1 \\ & y_1 + 3y_2 \geq 2 \\ & y_1, y_2 \geq 0\end{array}$$

Two structural constraints in the primal
become two dual *variables*; two primal
variables become two dual constraints.



$$w(2, 0) = 8 \quad w(0, 1) = 6 \quad w(0.5, 0.5) = 5 \star \quad \text{--- matches } z^* = 5: \text{ strong duality confirmed.}$$

Complementary Slackness — Checking the 2D Example

VERIFYING THE THEOREM ON x^* AND y^*

Recall $x^* = (3, 1)$, $z^* = 5$ and $y^* = (0.5, 0.5)$, $w^* = 5$ from the primal/dual example on the previous two slides.

Primal slack $\times$ dual variable

$$y_1^*(4 - (x_1^* + x_2^*)) = 0.5(4 - 4) = 0$$

$$y_2^*(6 - (x_1^* + 3x_2^*)) = 0.5(6 - 6) = 0$$

Both primal constraints are tight at x^* — so both products vanish.

Dual slack $\times$ primal variable

$$x_1^*((y_1^* + y_2^*) - 1) = 3(1 - 1) = 0$$

$$x_2^*((y_1^* + 3y_2^*) - 2) = 1(2 - 2) = 0$$

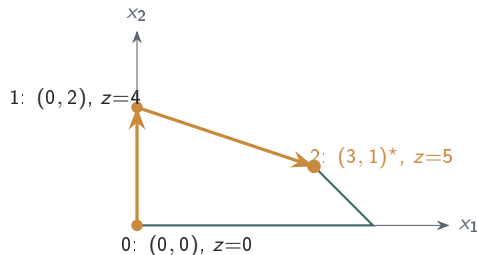
Both dual constraints are also tight at y^* — so these products vanish too.

All four complementary-slackness products are zero: the theorem holds. It does so in the strongest way here — every constraint is tight on both sides, since x^* and y^* are each pinned down by two intersecting active constraints in $\mathbb{R}^2$.

Simplex, Geometrically

INTRODUCING THE SIMPLEX ALGORITHM

Simplex walks from vertex to vertex along the edges of the polyhedron, always improving the objective, until no improving edge remains.



0→1 enter x_2 (largest reduced cost, 2); leave s_2 ;
ratio test $\min(\frac{4}{1}, \frac{6}{3}) = 2$

1→2 enter x_1 (reduced cost $\frac{1}{3}$); leave s_1 ; ratio
test $\min(\frac{2}{2/3}, \frac{2}{1/3}) = 3$

stop all reduced costs ≤ 0 : optimal

The Simplex Algorithm (Revised Form), Sketched

THE ALGORITHM BEHIND THE PICTURE

Standard form: maximize $c^\top x$ s.t. $Ax = b$, $x \geq 0$, with current basis B (an invertible $m \times m$ submatrix of A).

- 1 Compute $x_B = B^{-1}b$ (current basic feasible solution); nonbasic $x_N = 0$.
- 2 Compute reduced costs $\bar{c}_j = c_j - c_B^\top B^{-1}A_j$ for nonbasic j . If $\bar{c}_j \leq 0$ for all j : **optimal, stop**.
- 3 Choose an entering variable x_j with $\bar{c}_j > 0$ (Dantzig's rule: the largest one).
- 4 Compute the direction $d = B^{-1}A_j$.
- 5 If $d \leq 0$: **unbounded, stop**. Otherwise, ratio test: $t^* = \min_i \{(x_B)_i / d_i : d_i > 0\}$ fixes the leaving variable.
- 6 Update the basis (swap the leaving column for A_j); update B^{-1} , x_B , c_B . Return to step 2.

“Revised” because only B^{-1} (or its factorization) is carried between iterations — never a full tableau.

Two branches of discrete optimization

A CLOSER LOOK AT DISCRETE

Integer Programming

The discrete set is a subset of the integers. Variable may be binary (0/1) or general integers.

e.g. production quantities, on/off decisions, facility siting

Combinatorial Optimization

The discrete set is a set of objects, or combinatorial structures.

e.g. assignments, combinations, routes, schedules, sequences

Integer Programming — Cutting-Plane Methods

A CLOSER LOOK AT DISCRETE · SOLVING VIA CUTTING PLANES

Idea: repeatedly tighten the LP relaxation with extra linear inequalities (**cuts**) until an integer solution appears.

- 1 Solve the LP relaxation of the integer program.
- 2 If the optimal solution is already integer, **stop** — it solves the original problem too.
- 3 Otherwise, find a **valid inequality**: satisfied by every integer feasible point, but violated by the current fractional optimum.
- 4 Add the cut to the LP, resolve, and return to step 2.

Chvátal–Gomory cuts: a systematic source of valid inequalities, built directly from the constraints themselves — take a nonnegative combination of them, then round the right-hand side down (the left side is automatically integer-valued at any integer point).

Chvátal–Gomory Cut — Example

CUTTING OFF THE FRACTIONAL VERTEX (2.25, 2.25)

Recall the LP relaxation from a few slides back: its optimum (2.25, 2.25) is fractional.

Step 1 — combine. Add $\frac{1}{4}$ of each constraint:

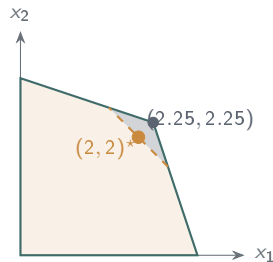
$$\begin{aligned}\frac{1}{4}(x_1 + 3x_2) + \frac{1}{4}(3x_1 + x_2) &\leq \frac{1}{4}(9) + \frac{1}{4}(9) \\ \implies x_1 + x_2 &\leq 4.5\end{aligned}$$

Step 2 — round. $x_1, x_2 \in \mathbb{Z} \Rightarrow x_1 + x_2 \in \mathbb{Z}$, so round the RHS down:

$$x_1 + x_2 \leq 4 \quad (\text{the cut})$$

Check: at (2.25, 2.25), $x_1 + x_2 = 4.5 > 4$ — cut off.

At (2, 2), $x_1 + x_2 = 4$: stays feasible, and is now the LP optimum too.



The dashed line is the cut $x_1 + x_2 = 4$: it slices off the shaded sliver containing the fractional vertex, without removing any integer point.

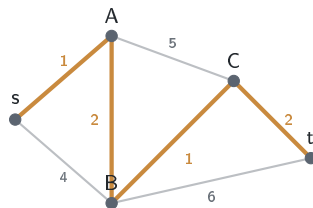
Combinatorial Optimization — Shortest Path

COMBINATORIAL OPTIMIZATION · GRAPHS

Problem. Given a weighted graph and two vertices s, t , find a path from s to t of minimum total edge weight.

e.g. GPS navigation, network routing, logistics.

Also polynomial: Dijkstra's algorithm, $O(E \log V)$, for nonnegative weights; Bellman–Ford handles negative weights.



Gold path $s \rightarrow A \rightarrow B \rightarrow C \rightarrow t$: total length $1+2+1+2 = 6$ — shorter than any other route (e.g. $s \rightarrow B \rightarrow t = 10$).

Shortest Path — Integer Formulation

FROM GRAPH TO INTEGER PROGRAM

For each directed edge $(i, j) \in E$, let $x_{ij} \in \{0, 1\}$ indicate whether it is used.

$$\begin{array}{ll}\text{minimize} & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{subject to} & \sum_{j:(s,j) \in E} x_{sj} - \sum_{j:(j,s) \in E} x_{js} = 1 \\ & \sum_{j:(v,j) \in E} x_{vj} - \sum_{j:(j,v) \in E} x_{jv} = 0, \quad \forall v \neq s, t \\ & \sum_{j:(t,j) \in E} x_{tj} - \sum_{j:(j,t) \in E} x_{jt} = -1 \\ & x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in E\end{array}$$

These are **flow-conservation** constraints: one unit of flow leaves s , passes through unchanged at every other vertex, and arrives at t . Unlike MST or TSP, no subtour constraints are needed — and remarkably, relaxing $x_{ij} \in \{0, 1\}$ to $x_{ij} \geq 0$ already gives an integer optimal solution: the constraint matrix is totally unimodular.

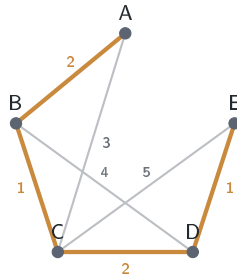
Combinatorial Optimization — Minimum Spanning Tree

COMBINATORIAL OPTIMIZATION · GRAPHS

Problem. Given a connected, weighted, undirected graph, find a subset of edges that connects all vertices, forms no cycle (a tree), and has minimum total weight.

e.g. network design: roads, pipelines, circuits, clustering.

Solvable in polynomial time (Kruskal's or Prim's algorithm, $O(E \log V)$) — despite being “combinatorial”, it is **easy**.



Gold edges: the minimum spanning tree (weight $1 + 1 + 2 + 2 = 6$). Muted edges exist in the graph but are not needed.

Minimum Spanning Tree — Integer Formulation

FROM GRAPH TO INTEGER PROGRAM

For each edge $e \in E$, let $x_e \in \{0, 1\}$ indicate whether e is chosen.

$$\begin{array}{ll}\text{minimize} & \sum_{e \in E} w_e x_e \\ \text{subject to} & \sum_{e \in E} x_e = |V| - 1 \\ & \sum_{e \in E(S)} x_e \leq |S| - 1, \quad \forall \emptyset \neq S \subsetneq V \\ & x_e \in \{0, 1\}, \quad \forall e \in E\end{array}$$

The first constraint fixes the number of edges to a tree's; the second (one per subset S , so **exponentially many**) forbids a cycle from forming inside any subset $E(S)$ of edges. Together they force a spanning tree. In practice these subtour constraints are added lazily, one violated cut at a time — the cutting-plane idea from a few slides back.

MST Subtour Constraints — The Separation Problem

CAN WE EVEN CHECK EXPONENTIALLY MANY CONSTRAINTS?

Separation problem. Given a point $x^* \in [0, 1]^E$ (e.g. the current LP solution), either confirm it satisfies every subtour constraint, or produce one that it violates:

$$\exists S \subsetneq V : \sum_{e \in E(S)} x_e^* > |S| - 1 \quad ?$$

This is exactly **submodular function minimization** in disguise (the map $S \mapsto x^*(E(S)) - |S|$ is submodular): it reduces to a polynomial number of **minimum-cut** computations on a graph with capacities x_e^* . So the separation problem for the subtour constraints is solvable in **polynomial time** — even though there are exponentially many of them, we never need to list them all.

Contrast with optimization: Kruskal's or Prim's algorithm directly finds the best spanning tree in polynomial time too — separation gives us a *second*, independent route to that same fact, one that generalizes far beyond MST.

Optimization = Separation

GRÖTSCHEL–LOVÁSZ–SCHRIJVER, 1981

For a well-described polyhedron P (rational, of polynomially bounded encoding size), the following are equivalent — each is solvable in polynomial time **if and only if** the other is:

- **Optimization:** given c , find $\max\{c^\top x : x \in P\}$.
- **Separation:** given x^* , decide whether $x^* \in P$, and if not, exhibit a violated inequality.

The **ellipsoid method** is the algorithmic bridge: it solves the optimization problem using only a separation oracle as a subroutine — it never needs the constraints written out, exponentially many or not.

Conclusion for MST: the subtour separation problem is polynomial (previous slide) $\Rightarrow$ by this theorem, the LP relaxation of the MST integer program — despite its exponentially many subtour constraints — can be **solved in polynomial time** via the ellipsoid method.

For MST, Integer Programming = Linear Programming

THE LP RELAXATION IS ALREADY INTEGRAL

Drop integrality in the MST formulation: replace $x_e \in \{0, 1\}$ with $x_e \geq 0$. Call the resulting polytope P_{ST} :

$$P_{ST} = \left\{ x \geq 0 : \sum_{e \in E} x_e = |V| - 1, \sum_{e \in E(S)} x_e \leq |S| - 1 \forall S \subsetneq V \right\}$$

Theorem (Edmonds, 1971). P_{ST} is **integral**: every vertex of P_{ST} is 0/1 — the incidence vector of an actual spanning tree. (P_{ST} is exactly the base polytope of the graphic matroid on E , and base polytopes of matroids are always integral.)

Consequence: solving the LP relaxation — which the previous two slides showed is polynomial — **already** returns an optimal spanning tree. No rounding, no branching: for MST, the integer program and its LP relaxation have the same optimal value *and* the same optimal solutions.

Contrast this with TSP a couple of slides ahead: its subtour polytope is **not** integral in general — the LP relaxation can have fractional vertices, which is part of why TSP stays NP-hard despite a similar-looking formulation.

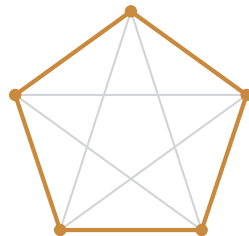
Combinatorial Optimization — Traveling Salesman

COMBINATORIAL OPTIMIZATION · GRAPHS

Problem. Given a weighted complete graph, find the minimum-weight Hamiltonian cycle: a closed tour visiting every vertex exactly once.

e.g. vehicle routing, logistics, circuit board drilling.

Unlike MST or shortest path, TSP is **NP-hard** — no known polynomial algorithm. With n cities there are $(n-1)!/2$ possible tours: already $> 10^{16}$ at $n = 20$.



Cities in convex position: the shortest tour is simply the polygon's boundary (gold) — any tour using a crossing pair of edges (muted) can always be shortened by uncrossing them.

Traveling Salesman — Integer Formulation

FROM GRAPH TO INTEGER PROGRAM

For each edge $\{i, j\}$ of the complete graph, let $x_{ij} \in \{0, 1\}$ indicate whether it is in the tour.

$$\begin{array}{ll}\text{minimize} & \sum_{i < j} w_{ij} x_{ij} \\ \text{subject to} & \sum_{j \neq i} x_{ij} = 2, \quad \forall i \in V \\ & \sum_{i, j \in S, i < j} x_{ij} \leq |S| - 1, \quad \forall S \subsetneq V, 2 \leq |S| \leq n-1 \\ & x_{ij} \in \{0, 1\}\end{array}$$

The degree constraints force exactly two tour edges at each city; the **subtour elimination** constraints (again exponentially many) rule out disjoint smaller loops, forcing one single cycle through all n cities. This is the classical Dantzig–Fulkerson–Johnson formulation — solved in practice by adding violated subtour cuts one at a time, exactly like the cutting-plane method seen earlier.

Branch-and-Bound — The General Idea

THE OTHER CLASSIC WAY TO SOLVE INTEGER PROGRAMS

Idea: implicitly enumerate every integer feasible solution by organizing the search as a tree, and prune branches that provably cannot contain a better solution.

Branching

Pick a variable with a fractional value in the current LP relaxation, say $x_j = v \notin \mathbb{Z}$, and split the problem in two:

$$x_j \leq \lfloor v \rfloor \quad \text{and} \quad x_j \geq \lceil v \rceil$$

Every integer feasible solution of the parent lies in exactly one child — nothing is lost.

Bounding

Solve the LP relaxation of a subproblem to get a bound on its best possible value. Compare against the **incumbent** (best integer solution found so far) and discard (*fathom*) the subproblem if:

- it is infeasible, or
- its bound cannot beat the incumbent, or
- its LP solution is already integer (nothing left to branch on).

Branch-and-Bound — Algorithm Sketch

THE LOOP BEHIND THE TREE

Maintain a list of open subproblems and an incumbent value z^* (initially $-\infty$ for a maximization).

- ❶ If the list is empty, **stop**: the incumbent is optimal.
- ❷ Pick a subproblem P from the list.
- ❸ Solve the LP relaxation of P , giving a bound z_{LP} .
 - **Infeasible**: fathom P (discard it).
 - $z_{LP} \leq z^*$: fathom P — it cannot improve on what we already have.
 - LP solution is integer: update $z^* \leftarrow z_{LP}$ if better, then fathom P .
 - Otherwise: **branch** on a fractional variable, add the two children to the list.
- ❹ Return to step 1.

The LP relaxation only ever needs to be solved, never the integer program itself directly — branch-and-bound turns repeated LP solves into an exact algorithm for the integer program.

Branch-and-Bound — Worked Example

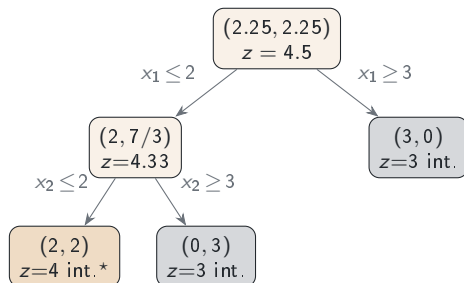
BACK TO THE 2D INTEGER PROGRAM

Root LP relaxation: $(2.25, 2.25)$, $z = 4.5$ — fractional. Branch on x_1 .

$x_1 \leq 2$: LP optimum $(2, 7/3)$, $z = 13/3 \approx 4.33$ — still fractional. Branch on x_2 .

- $x_2 \leq 2$: LP optimum $(2, 2)$, $z = 4$ — **integer**, new incumbent.
- $x_2 \geq 3$: LP optimum $(0, 3)$, $z = 3$ — integer, worse.

$x_1 \geq 3$: forces $(3, 0)$, $z = 3$ — integer, worse.



Every leaf is integer, so every branch is fathomed. Best found: $(2, 2)^*$, $z^* = 4$ — matching the true integer optimum.

Deterministic vs. Stochastic

AXIS 4 · DATA YOU CAN'T PIN DOWN

Deterministic optimization assumes the data are known accurately. In practice, future demand, prices and other inputs often can't be known with certainty — two frameworks handle that explicitly.

	Stochastic Programming	Robust Optimization
Models uncertainty as	Random variables with known (or estimated) probability distributions	Set membership — parameters known only within certain bounds
Optimizes	The expected performance across possible data instances	The worst possible case, feasible for all the data

Stochastic Programming — The Newsvendor Problem

AXIS 4 · A CLASSIC TWO-STAGE STOCHASTIC PROGRAM

A vendor must choose an order quantity q **before** observing demand D , a random variable with known distribution. Two costs trade off:

- **Overage cost** c_o : paid per unit left unsold, if $q > D$.
- **Underage cost** c_u : paid per unit of unmet demand, if $q < D$.

$$\min_{q \geq 0} \mathbb{E} [c_o (q - D)^+ + c_u (D - q)^+]$$

A two-stage stochastic program: q is the here-and-now (first-stage) decision; the recourse cost above is paid only once D is revealed (second-stage, wait-and-see).

Newsvendor — An Intuitive Argument First

SHOULD I ORDER ONE MORE UNIT?

Think about the **marginal** unit: should we increase the order quantity from q to $q + 1$?

- If $D \leq q$ (this happens with probability $F(q)$): that extra unit is **not sold** — it costs us c_o for nothing.
- If $D > q$ (probability $1 - F(q)$): that extra unit **is sold** — it saves us one unit of shortage, worth c_u .

Expected cost of ordering one more unit:

$$\underbrace{c_o F(q)}_{\text{expected overage cost}} - \underbrace{c_u (1 - F(q))}_{\text{expected shortage avoided}}$$

As long as this quantity is **negative**, the extra unit reduces expected cost on average — keep raising q . The optimal q^* is where it hits **zero**: increasing further no longer pays off.

Newsvendor — The Formal Derivation

THE CRITICAL FRACTILE

Write the expected cost as an integral over the density f of D (with F its CDF):

$$\mathbb{E}[\text{cost}(q)] = c_o \underbrace{\int_0^q (q-d) f(d) dd}_{\text{overage term}} + c_u \underbrace{\int_q^\infty (d-q) f(d) dd}_{\text{underage term}}$$

Differentiate each term with respect to q (the boundary contribution vanishes in both cases, since $q-d=0$ at $d=q$):

$$\frac{d}{dq} \left[c_o \int_0^q (q-d) f(d) dd \right] = c_o F(q), \quad \frac{d}{dq} \left[c_u \int_q^\infty (d-q) f(d) dd \right] = -c_u (1 - F(q))$$

Setting the sum — exactly the marginal quantity from the previous slide — to zero:

$$c_o F(q) - c_u (1 - F(q)) = 0 \implies F(q^*) = \frac{c_u}{c_u + c_o}$$

$$q^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right)$$

Second derivative $= (c_o + c_u)f(q) \geq 0$: the expected cost is convex in q , so this stationary point is indeed the global minimum.

Newsvendor — Worked Example

PUTTING NUMBERS ON IT

Demand $D \sim \mathcal{N}(100, 20^2)$. Unit cost $c = \$4$, selling price $p = \$10$, salvage value $s = \$2$ for leftover units.

$$c_u = p - c = 6 \quad (\text{lost profit per unit short})$$

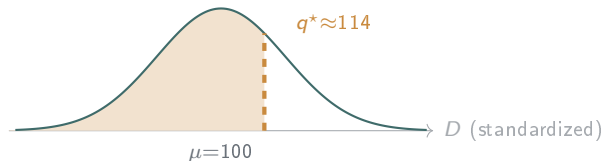
$$c_o = c - s = 2 \quad (\text{loss per unit unsold})$$

Critical ratio:

$$\frac{c_u}{c_u + c_o} = \frac{6}{8} = 0.75$$

Order quantity:

$$q^* = 100 + 20 \Phi^{-1}(0.75) \approx 114$$



Shaded area = 0.75: order enough to cover 75% of possible demand outcomes — underage is $3\times$ costlier than overage, so it pays to lean toward overstocking.

Stochastic Programming — Optimal “On Average”

WHAT DOES “OPTIMAL” EVEN MEAN UNDER UNCERTAINTY?

The newsvendor’s q^* does **not** guarantee the best outcome for any single realization of demand D — it guarantees the best **expected** outcome, averaged over every possible realization, weighted by its probability.

No other order quantity q achieves a lower $\mathbb{E}[\text{cost}]$ — but for one specific realized D , some other q (chosen with hindsight) would almost always have done better.

Example. With $q^* \approx 114$:

- If $D = 80$: 34 units unsold — overage cost paid, a “bad” outcome in hindsight.
- If $D = 140$: 26 units of unmet demand — underage cost paid, also “bad” in hindsight.
- Yet across the *whole distribution* of D , q^* minimizes the average cost — no fixed q does better.

This “on average” guarantee fits decisions that repeat many times (law of large numbers), or a risk-neutral decision-maker. It is a very different promise from robust optimization’s worst-case guarantee, which protects every single realization at the cost of average performance.

Robust Optimization — The Same Newsvendor, Robustified

AXIS 4 · NO DISTRIBUTION, JUST A RANGE

Now suppose we don't trust a probability distribution for D — we only know it lies in an interval $D \in [D_{\min}, D_{\max}]$. Robust optimization protects against the **worst case** in that range:

$$\min_{q \geq 0} \max_{D \in [D_{\min}, D_{\max}]} \left[c_o(q - D)^+ + c_u(D - q)^+ \right]$$

The inner cost, as a function of D alone, is convex (“V-shaped”: it falls then rises around $D = q$). The maximum of a convex function over an interval is always at one of its **endpoints** — so the worst case is always either “demand turns out to be as low as possible” or “as high as possible,” never something in between:

$$\max_{D \in [D_{\min}, D_{\max}]} [\dots] = \max \{ c_o(q - D_{\min}), c_u(D_{\max} - q) \}$$

Robust Newsvendor — Balancing the Worst Case

A MINIMAX, BALANCED AT THE CROSSING POINT

One term grows with q , the other shrinks with q : the best (smallest) worst case is where they **cross**:

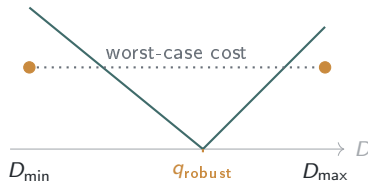
$$c_o(q - D_{\min}) = c_u(D_{\max} - q)$$

$$q_{\text{robust}} = \frac{c_u D_{\max} + c_o D_{\min}}{c_u + c_o}$$

With $c_u = 6$, $c_o = 2$ and $D \in [60, 140]$:

$$q_{\text{robust}} = \frac{6(140) + 2(60)}{8} = 120$$

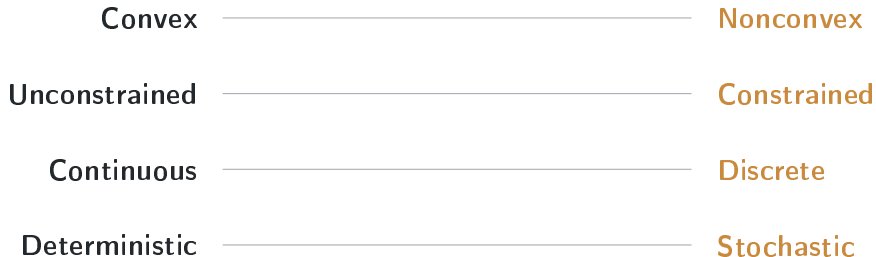
Worst-case cost = 120, guaranteed for **every** $D \in [60, 140]$ — versus $q^* \approx 114$ from the stochastic version, only best *on average*.



The V-shaped cost in D at q_{robust} : both endpoints reach exactly the same height — neither extreme is worse than the other.

Locate your model on all four axes

PUTTING IT TOGETHER



A real model is a point on all four lines at once — and that point decides which algorithm family applies.